Mediahub

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für IT-Medientechnik

Eingereicht von:
Markus Tran
Abdulkerim Cufurovic

Betreuer:
Prof. Mag. Robert Reder

Leonding, 4. April 2025

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 4. April 2025 Markus Tran & Abdulkerim Cufurovic

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Motivation und Problemstellung der Abo-Verwaltung 1

1.2 Zielsetzung der Diplomarbeit 1

1.3 Nutzen der Anwendung für Endanwender 1

1.4 Aufbau der Diplomarbeit 1

2 Ausgangssituation und Analyse **2**

2.1 Aktuelle Herausforderungen bei Streaming-Abonnements 2

2.2 Probleme durch parallele Abos und doppelte Abbuchungen 2

2.3 Bestehende Lösungen und deren Schwächen 2

2.4 Abgrenzung zu Standard-Webprojekten 3

3 Fachlicher Hintergrund Abo- & Mediatheken-Management **4**

3.1 Zentrale Verwaltung von Abonnements und Laufzeiten 4

3.2 Darstellung von Verfügbarkeiten von Filmen und Serien 4

3.3 Mehrwert einer zentralen Applikation für Nutzer 4

3.4 Datenschutz- und Sicherheitsanforderungen 5

4 Technologische Grundlagen **6**

4.1 Angular 6

4.2 Keycloak 7

4.3 Tailwind CSS 9

4.4 GitHub 10

4.5 Quarkus – Grundlagen und Einsatz in der Anwendung 11

4.6 Grundprinzip von Quarkus 11

4.7 Quarkus in der vorliegenden Applikation 13

4.8 Keycloak 13

5 Architektur und Systemdesign **14**
5.1 Gesamtarchitektur der Anwendung 14
5.2 Kommunikation zwischen Frontend und Backend 15
5.3 API-Integration externer Movie-Datenbanken 15
5.4 Sicherheits- und Datenschutzkonzept 16
5.5 Datenbanken 17
5.6 Datenbankkonzept und architektonische Entscheidungen 18
5.7 Backend-Architektur 22
5.8 Implementierung des Backends in der vorliegenden Applikation 24

6 Umsetzung der Anwendung **30**
6.1 Implementierung der Abo-Verwaltung 30
6.2 Darstellung von Laufzeiten und Kündigungszeitpunkten 31
6.3 Verknüpfung von Filmen mit Streaming-Anbieter 32
6.4 Account- und Passwortverwaltung 34
6.5 Herausforderungen und Reflexion der Frontend-Entwicklung 36

7 Projektverlauf und Entscheidungen **38**
7.1 Projektplanung und Arbeitsweise 38
7.2 Technische und fachliche Herausforderungen 38
7.3 Entscheidungsfindung im Projektteam 38
7.4 Einbindung von Betreuern und Feedback 38
7.5 Reaktion auf Fragen und Kritik aus Präsentationen 38

8 Ergebnisse und Evaluation **39**
8.1 Erfüllung der gesetzten Ziele 39
8.2 Bewertung des Nutzens für Endanwender 39
8.3 Reflexion der eigenen Leistung 39

9 Ausblick **40**
9.1 Mögliche Erweiterungen der Anwendung 40
9.2 Zukunftspotenzial der Plattform 40

10 Zusammenfassung **41**

11 Quellen-, Literatur- und Abbildungsverzeichnis **42**
11.1 Quellen 42
11.2 Literatur 42

11.3 Abbildungsverzeichnis 42

Literaturverzeichnis **V**

Abbildungsverzeichnis **VI**

Tabellenverzeichnis **VII**

Quellcodeverzeichnis **VIII**

1 Einleitung

1.1 Motivation und Problemstellung der Abo-Verwaltung

Text ...

1.2 Zielsetzung der Diplomarbeit

Text ...

1.3 Nutzen der Anwendung für Endanwender

Text ...

1.4 Aufbau der Diplomarbeit

Text ...

2 Ausgangssituation und Analyse

2.1 Aktuelle Herausforderungen bei Streaming-Abonnements

Streaming-Nutzer stehen heute vor der Herausforderung, dass Inhalte auf viele Anbieter verteilt sind und sich gleichzeitig die Preisstruktur laufend verändert. Laut aktuellen Erhebungen haben viele Haushalte mehrere parallele Streaming-Abonnements, während gleichzeitig die Zahlungsbereitschaft sinkt, wenn Preise steigen oder Inhalte als wenig relevant wahrgenommen werden [1]. Dadurch entsteht ein praktisches Problem, das über die reine Unterhaltung hinausgeht: die bewusste Verwaltung begrenzter Medienbudgets.

2.2 Probleme durch parallele Abos und doppelte Abbuchungen

Wer mehrere Dienste gleichzeitig nutzt, verliert leicht den Überblick über laufende Kosten, Abrechnungszyklen und aktive Plattformen. Besonders problematisch wird dies, wenn Dienste selten verwendet werden, aber weiterhin verrechnet werden. Eine Anwendung, die Anbieter, Abrechnungsdaten und persönliche Auswahl in einer Oberfläche bündelt, kann genau an dieser Stelle ansetzen und Transparenz schaffen.

2.3 Bestehende Lösungen und deren Schwächen

Bestehende Angebote konzentrieren sich oft entweder auf allgemeine Finanzverwaltung oder auf die reine Anzeige von Filmverfügbarkeiten. Eine Kombination aus Filmsuche, Anbieteranzeige und persönlicher Abo-Verwaltung ist in dieser Form deutlich seltener. Genau diese Lücke war für das Projekt zentral, da nicht nur Daten angezeigt, sondern für den konkreten Medienalltag nutzbar gemacht werden sollten.

2.4 Abgrenzung zu Standard-Webprojekten

Im Unterschied zu einem klassischen Informations- oder Demo-Webprojekt musste hier nicht nur eine Oberfläche erstellt werden. Vielmehr war eine Anwendung zu entwickeln, die externe Filmdaten, Authentifizierung, benutzerspezifische Speicherung und verwaltbare Abonnements sinnvoll zusammenführt. Gerade die Verknüpfung zwischen allgemeiner Medieninformation und persönlicher Nutzungssituation macht den fachlichen Anspruch des Projekts aus.

3 Fachlicher Hintergrund Abo- & Mediatheken-Management

3.1 Zentrale Verwaltung von Abonnements und Laufzeiten

Mit der steigenden Zahl digitaler Streaming-Dienste wird auch die Verwaltung paralleler Abonnements komplexer. Nutzerinnen und Nutzer verwenden heute oft mehrere kostenpflichtige Angebote gleichzeitig, wodurch Monatskosten, Abrechnungszeitpunkte und inhaltliche Überschneidungen schwerer nachvollziehbar werden. Aktuelle Branchenanalysen zeigen, dass Haushalte regelmäßig mehrere Streaming-Abonnements parallel verwenden und dabei zunehmend auf Preis-Leistungs-Verhältnis, Nutzungsintensität und Kündigungsflexibilität achten [1].

3.2 Darstellung von Verfügbarkeiten von Filmen und Serien

Neben der reinen Verwaltung von Abonnements ist für Anwender vor allem relevant, auf welcher Plattform ein bestimmter Film oder eine Serie verfügbar ist. Genau an dieser Stelle entsteht ein praktischer Mehrwert, wenn Inhalte und persönliche Abonnements gemeinsam betrachtet werden. Die verwendete TMDB API stellt dafür strukturierte Endpunkte für Filmdaten, Metadaten und Anbieterinformationen bereit und bildete damit eine fachliche Grundlage für die Umsetzung des Projekts [2, 3].

3.3 Mehrwert einer zentralen Applikation für Nutzer

Eine zentrale Applikation für Medien- und Abo-Management reduziert den organisatorischen Aufwand für Benutzer deutlich. Statt verschiedene Streaming-Plattformen einzeln

zu überprüfen, lassen sich Suchfunktion, Verfügbarkeitsanzeige und eigene Abo-Daten in einer Anwendung zusammenführen. Gerade in einem Markt mit wachsender Anzahl an Diensten und steigender Preissensibilität wird ein solches zentrales Werkzeug für Benutzer zunehmend sinnvoll [1].

3.4 Datenschutz- und Sicherheitsanforderungen

Sobald eine Anwendung persönliche Nutzungsdaten, Abonnements oder geschützte Funktionen bereitstellt, entstehen zusätzliche Anforderungen an Authentifizierung und Zugriffsschutz. Die Trennung zwischen öffentlichen Film- und Providerdaten einerseits und benutzerspezifischen Abo-Daten andererseits ist deshalb nicht nur eine technische, sondern auch eine fachliche Notwendigkeit. Für das Projekt war daraus abzuleiten, dass Benutzerkonten, Rollen und geschützte Requests von Beginn an als eigener Sicherheitsbereich betrachtet werden müssen.

4 Technologische Grundlagen

4.1 Angular

Im Rahmen meines Projektteils lag der Schwerpunkt auf der Entwicklung des Frontends. Als technologische Grundlage wurde dafür Angular verwendet, da das Framework eine klare Struktur für moderne Webanwendungen bietet und zentrale Funktionen wie Routing, Komponentenarchitektur, Formularverarbeitung und HTTP-Kommunikation bereits integriert [4, 5, 6]. Für die Umsetzung des MedienAbonnementManagers war Angular besonders geeignet, da mehrere zusammenhängende Oberflächenbereiche entwickelt werden mussten, die dennoch in einer einheitlichen Anwendung miteinander verbunden bleiben sollten.

4.1.1 Rolle von Angular innerhalb der Anwendung

Angular übernimmt in der Anwendung die gesamte Darstellung und Interaktion der Benutzeroberfläche. Dazu zählen insbesondere die Startseite, die Filmsuche, die Detailansicht einzelner Filme, die Navigation innerhalb der Anwendung sowie die Oberfläche zur Verwaltung persönlicher Streaming-Abonnements. Der Fokus meines Projektteils lag darauf, diese Funktionen so umzusetzen, dass sie übersichtlich, wartbar und für Benutzer intuitiv bedienbar sind.

4.1.2 Architektur als Single-Page-Application

Das Frontend wurde als Single-Page-Application umgesetzt. Bei diesem Ansatz bleibt die Anwendung nach dem ersten Laden im Browser aktiv und Inhalte werden dynamisch aktualisiert, ohne dass bei jeder Aktion ein vollständiger Seitenneuaufbau erfolgen muss [7]. Diese Architektur verbessert die Benutzererfahrung deutlich, da Navigationsvorgänge zwischen Startseite, Filmsuche, Detailansicht und Providerverwaltung flüssig ablaufen. Gerade für eine interaktive Webanwendung mit mehreren Ansichten stellt dies einen wesentlichen Vorteil dar.

4.1.3 Begründung der Wahl von Angular gegenüber React

Für dieses Projekt wurde Angular gewählt, weil es ein vollständiges Framework mit klarer Struktur und fest integrierten Kernfunktionen darstellt [4, 8]. Im Vergleich zu React bietet Angular bereits standardisierte Lösungen für Routing, Formulare und HTTP-Kommunikation. Für den Frontend-Teil der Diplomarbeit war dies besonders hilfreich, da dadurch eine einheitliche Entwicklung möglich war und weniger zusätzliche Bibliotheken benötigt wurden. Außerdem erleichtert Angular durch seinen komponentenbasierten Aufbau die langfristige Wartung und Erweiterung der Anwendung.

4.1.4 Komponentenstruktur der Anwendung

Die Benutzeroberfläche wurde in mehrere eigenständige Angular-Komponenten gegliedert. Dazu gehören unter anderem die Hauptseite, die Filmübersicht, die Detailansicht eines Films, die Providerverwaltung, die Navigationsleiste sowie die Ansichten für Login und Registrierung. Diese Aufteilung ermöglicht eine saubere Trennung einzelner Funktionsbereiche. Änderungen an einer Komponente können dadurch umgesetzt werden, ohne andere Bereiche der Anwendung unnötig zu beeinflussen.

4.1.5 Data Binding und Services zur Zustandsverwaltung

Ein wesentlicher Bestandteil der Frontend-Entwicklung war die Verknüpfung von Benutzereingaben mit den dargestellten Daten. Angular stellt dafür Mechanismen wie Data Binding und Formularunterstützung bereit [6]. Im Projekt wurden außerdem eigene Services eingesetzt, um Datenzugriffe zu kapseln und die Kommunikation mit den bereitgestellten Schnittstellen zu organisieren. Dadurch bleibt die Oberflächenlogik übersichtlich und die einzelnen Komponenten konzentrieren sich auf Darstellung und Interaktion.

4.2 Keycloak

Für die Anmeldung und Registrierung der Benutzer wurde im Frontend Keycloak eingebunden. Dadurch musste keine eigene Login-Oberfläche mit selbst implementierter Passwortlogik entwickelt werden. Stattdessen wurde eine etablierte Lösung für Authentifizierung verwendet, die das Frontend mit dem Anmeldesystem verbindet [9, 10]. Aus

Sicht meines Projektteils war vor allem die Einbindung in die Benutzeroberfläche und die Steuerung geschützter Frontend-Bereiche relevant.

4.2.1 Grundlagen der Authentifizierung aus Frontend-Sicht

Im Frontend dient die Authentifizierung dazu, zwischen öffentlichen und geschützten Bereichen der Anwendung zu unterscheiden. Öffentliche Inhalte wie die Filmübersicht können allgemein angezeigt werden, während persönliche Bereiche wie die Verwaltung eigener Abonnements nur angemeldeten Benutzern zugänglich sind. Das Frontend reagiert damit auf den Authentifizierungsstatus und passt Navigation sowie verfügbare Funktionen entsprechend an.

4.2.2 Ablauf des Login- und Token-Prozesses

Beim Login wird der Benutzer aus dem Frontend heraus an Keycloak weitergeleitet. Nach erfolgreicher Anmeldung kehrt er in die Anwendung zurück und kann geschützte Funktionen nutzen [9]. Für das Frontend ist dabei entscheidend, dass der Benutzerstatus erkannt und für weitere Anfragen verwendet wird. Auf diese Weise können geschützte Ansichten nur dann geöffnet werden, wenn eine gültige Anmeldung vorliegt.

4.2.3 Rollen und Zugriff auf geschützte Bereiche

Neben der reinen Anmeldung spielt auch die Zugriffskontrolle auf bestimmte Ansichten eine Rolle. Im Frontend wird geprüft, ob ein Benutzer auf geschützte Bereiche wie die Providerverwaltung zugreifen darf. Dadurch kann die Navigation benutzerfreundlich gestaltet werden, indem nicht erlaubte Seiten entweder verborgen oder nur nach erfolgreicher Anmeldung erreichbar gemacht werden.

4.2.4 Nutzen für Sicherheit und Benutzerfreundlichkeit

Die Verwendung von Keycloak verbessert sowohl die Sicherheit als auch die Benutzerfreundlichkeit. Da keine eigene Passwortverwaltung entwickelt werden musste, konnte das Risiko unsicherer Eigenimplementierungen reduziert werden. Gleichzeitig wirkt der Login-Prozess für Benutzer konsistent, da Registrierung, Anmeldung und Abmeldung zentral über ein spezialisiertes System gesteuert werden.

4.3 Tailwind CSS

Für die Gestaltung der Benutzeroberfläche wurde Tailwind CSS eingesetzt. Tailwind ist ein CSS-Framework, das nach dem Utility-First-Prinzip arbeitet und dadurch eine schnelle und flexible Entwicklung moderner Benutzeroberflächen ermöglicht [11, 12]. Im Frontend des MedienAbonnementManagers wurde Tailwind genutzt, um Layout, Abstände, Farben, Formulare und Kartenansichten effizient zu gestalten.

4.3.1 Utility-First-Ansatz

Der Utility-First-Ansatz bedeutet, dass viele kleine CSS-Klassen direkt in den Templates verwendet werden, um konkrete Gestaltungseigenschaften festzulegen [11]. Dazu zählen unter anderem Abstände, Größen, Farben, Rahmen oder Typografie. Für das Projekt hatte dies den Vorteil, dass Oberflächen schnell angepasst und konsistent gestaltet werden konnten, ohne für jede Änderung umfangreiche eigene CSS-Strukturen anlegen zu müssen.

4.3.2 Gestaltung der Benutzeroberfläche

Tailwind CSS wurde besonders für interaktive Bereiche wie die Providerverwaltung, Formulare, Suchfelder, Buttons und Kartenansichten eingesetzt. Dadurch konnte eine moderne und übersichtliche Oberfläche umgesetzt werden, die die verschiedenen Funktionsbereiche visuell voneinander trennt. Vor allem in der Abo-Verwaltung war eine klare Darstellung wichtig, damit Benutzer Anbieter, Status und Eingabefelder schnell erfassen können.

4.3.3 Integration von Tailwind CSS in Angular

Die Einbindung von Tailwind CSS in das Angular-Projekt erfolgte direkt über die Konfiguration des Frontends. Dadurch standen die Utility-Klassen in allen Komponenten zur Verfügung. Diese enge Integration war für die praktische Frontend-Entwicklung besonders hilfreich, da Layout und Funktionalität parallel umgesetzt werden konnten.

4.3.4 Beitrag zur Benutzerfreundlichkeit

Ein zentrales Ziel des Frontends war eine intuitive und ansprechende Bedienung. Die Gestaltung mit Tailwind CSS unterstützt dieses Ziel, indem wichtige Elemente klar hervorgehoben und logisch angeordnet werden. Eine gut strukturierte Oberfläche trägt wesentlich dazu bei, dass Benutzer die Anwendung leichter verstehen und effizient verwenden können.

4.4 GitHub

GitHub wurde im Projekt als Plattform für Versionsverwaltung und Zusammenarbeit genutzt. Für meinen Frontend-Teil war dies insbesondere wichtig, um Änderungen an Komponenten, Styles und Routing nachvollziehbar zu dokumentieren und in einem strukturierten Entwicklungsprozess festzuhalten [13]. Dadurch konnten einzelne Entwicklungsschritte sauber voneinander getrennt und schrittweise weiterentwickelt werden.

4.4.1 Versionsverwaltung im Frontend

Die Frontend-Entwicklung umfasste mehrere aufeinander aufbauende Änderungen, beispielsweise an der Filmsuche, an der Navigationsstruktur, an der Provideransicht und am visuellen Design. Durch die Versionsverwaltung konnten diese Schritte einzeln dokumentiert und bei Bedarf auf frühere Stände zurückgeführt werden. Dies erhöhte die Nachvollziehbarkeit und reduzierte das Risiko, funktionierende Zwischenstände zu verlieren.

4.4.2 Branching und strukturierte Weiterentwicklung

Im Projekt wurden verschiedene Branches verwendet, um neue Änderungen getrennt vom Hauptstand weiterzuentwickeln. Für die Frontend-Arbeit war dies besonders hilfreich, wenn größere Überarbeitungen an der Benutzeroberfläche oder an der Logik einzelner Ansichten vorgenommen wurden. So konnten neue Ideen und Anpassungen zunächst isoliert entwickelt werden, bevor sie in einen stabileren Stand übernommen wurden.

4.4.3 Nachvollziehbarkeit der eigenen Arbeit

Die Commit-Historie dokumentiert die schrittweise Weiterentwicklung des Frontends, etwa bei der Überarbeitung der Filmsuche, beim Ausbau der Trending-Ansichten oder bei der Verbesserung der Benutzeroberfläche der Abo-Verwaltung. Für die Diplomarbeit ist dies hilfreich, da der eigene Beitrag am Frontend dadurch nachvollziehbar beschrieben werden kann.

4.5 Quarkus – Grundlagen und Einsatz in der Anwendung

Quarkus ist ein modernes, quelloffenes Java-Framework, das speziell für den Einsatz in containerisierten und Cloud-nativen Umgebungen entwickelt wurde. Ziel des Frameworks ist es, den Prozess der Bereitstellung von Java-Applikationen in Cloud-Umgebungen zu vereinfachen und zu optimieren. Dabei unterstützt Quarkus Entwicklerinnen und Entwickler dabei, Applikationen auch ohne tiefgehende Kenntnisse der zugrundeliegenden Infrastruktur bereitzustellen.

Ein wesentliches Merkmal von Quarkus ist seine Kompatibilität mit einer Vielzahl etablierter Frameworks und Standards. Dazu zählen unter anderem Eclipse MicroProfile, RESTEasy (JAX-RS), Hibernate ORM (JPA), Apache Kafka sowie Spring. Diese breite Unterstützung ermöglicht es, bestehende Kenntnisse und Bibliotheken direkt einzusetzen, ohne grundlegende Umstrukturierungen vornehmen zu müssen.

Quarkus hebt sich durch seine Leichtgewichtigkeit von anderen Java-Frameworks ab. Konkret äußert sich dies in einer geringen Startzeit, einem niedrigen Speicherverbrauch sowie der einfachen Möglichkeit, Projekte in Cloud-Umgebungen bereitzustellen.

4.6 Grundprinzip von Quarkus

Ein zentrales Merkmal von Quarkus liegt in der Art, wie Java-Anwendungen kompiliert und ausgeführt werden. Um dies zu verstehen, ist es notwendig, die zwei grundlegenden Kompilierungsansätze zu unterscheiden: die Just-In-Time-Kompilierung (JIT) und die Ahead-of-Time-Kompilierung (AOT).

4.6.1 Just-In-Time-Kompilierung (JIT)

Im klassischen Java-Ausführungsmodell wird Quellcode zunächst vom Java-Compiler in plattformunabhängigen Bytecode übersetzt. Dieser Bytecode wird anschließend von der Java Virtual Machine (JVM) interpretiert. Die JIT-Kompilierung ist dabei ein Optimierungsmechanismus der JVM: Häufig ausgeführte Codeabschnitte werden zur Laufzeit in nativen Maschinencode übersetzt, was die Ausführungsgeschwindigkeit erhöht.

Der Nachteil dieses Ansatzes liegt darin, dass die JVM beim Start der Applikation zunächst hochgefahren werden muss und die JIT-Optimierungen erst schrittweise während der Laufzeit greifen. Dies führt zu längeren Startzeiten und einem erhöhten Speicherverbrauch – Faktoren, die in containerisierten Umgebungen mit häufigen Neustarts besonders nachteilig sind.

4.6.2 Ahead-of-Time-Kompilierung (AOT)

Die Ahead-of-Time-Kompilierung verfolgt einen grundlegend anderen Ansatz: Der gesamte Anwendungscode wird bereits vor der Ausführung, also zum Zeitpunkt des Build-Prozesses, in nativen Maschinencode übersetzt. Quarkus nutzt hierfür GraalVM Native Image, ein Werkzeug, das Java-Anwendungen in plattformspezifische, ausführbare Binärdateien kompiliert.

Das Ergebnis ist eine native ausführbare Datei, die ohne eine laufende JVM auskommt. Dies hat folgende Vorteile:

- **Schnelle Startzeit:** Da keine JVM initialisiert werden muss und alle Optimierungen bereits zur Build-Zeit vorgenommen wurden, startet die Anwendung in Millisekunden.
- **Geringer Speicherverbrauch:** Der Wegfall der JVM-Laufzeitumgebung reduziert den RAM-Bedarf erheblich.
- **Eignung für containerisierte Umgebungen:** Kurze Startzeiten und geringer Ressourcenverbrauch machen AOT-kompilierte Anwendungen besonders geeignet für Kubernetes und ähnliche Plattformen.

Als Nachteil ist die deutlich längere Build-Zeit im Vergleich zur JIT-Kompilierung zu nennen, da die gesamte Kompilierung und Optimierung vorab stattfindet.

4.6.3 Hot Reload

Für den Entwicklungsprozess bietet Quarkus einen sogenannten Hot-Reload-Mechanismus, bekannt als *Quarkus Dev Mode*. Dabei werden Änderungen am Quellcode automatisch erkannt und ohne Neustart der Anwendung übernommen. Dies beschleunigt den Entwicklungszyklus erheblich, da zeitaufwändige Neustarts entfallen. Der Dev Mode verwendet intern JIT-Kompilierung, um diese schnelle Rückkopplungsschleife zu ermöglichen – die AOT-Kompilierung wird lediglich für den finalen Build eingesetzt.

4.7 Quarkus in der vorliegenden Applikation

Quarkus wird in der vorliegenden Anwendung als Hauptframework für das Backend verwendet. Für die Datenbankanbindung kommt Hibernate ORM mit Panache zum Einsatz, das eine vereinfachte Abstraktion des JPA-Standards bietet und den Datenbankzugriff mit weniger Boilerplate-Code ermöglicht.

Die Anwendung wird mittels AOT-Kompilierung als native ausführbare Datei gebaut. Obwohl die Build-Zeit dadurch im Vergleich zur klassischen JIT-Kompilierung länger ausfällt, überwiegen die Vorteile im Betrieb: Die Applikation weist eine deutlich verkürzte Startzeit auf, benötigt weniger Arbeitsspeicher und ist optimal für den Einsatz in einer containerisierten Umgebung geeignet. Diese Eigenschaften sind insbesondere in Cloud-Deployments von Vorteil, wo Ressourceneffizienz und Skalierbarkeit zentrale Anforderungen darstellen.

4.8 Keycloak

Um die Sicherheit der Authentifizierungs- und Autorisierungsanforderungen der Anwendung zu erfüllen, wurde Keycloak als Identity- und Access-Management-Lösung integriert. Keycloak bietet eine umfassende Plattform für die Verwaltung von Benutzern, Rollen und Berechtigungen sowie die Unterstützung verschiedener Authentifizierungsprotokolle wie OpenID Connect, OAuth 2.0 und SAML.

5 Architektur und Systemdesign

5.1 Gesamtarchitektur der Anwendung

Die Gesamtarchitektur der Anwendung folgt einem klar getrennten Aufbau zwischen Benutzeroberfläche und serverseitiger Verarbeitung. Der Schwerpunkt meines Projektteils lag auf dem Frontend, das mit Angular als Single-Page-Application umgesetzt wurde. Das Frontend übernimmt die Darstellung der Inhalte, die Navigation zwischen den Ansichten sowie die Interaktion mit den Benutzern. Die eigentliche Datenverarbeitung erfolgt über bereitgestellte REST-Schnittstellen, die vom Frontend angesprochen werden. Durch diese Trennung konnte die Benutzeroberfläche unabhängig von der serverseitigen Implementierung entwickelt und erweitert werden.

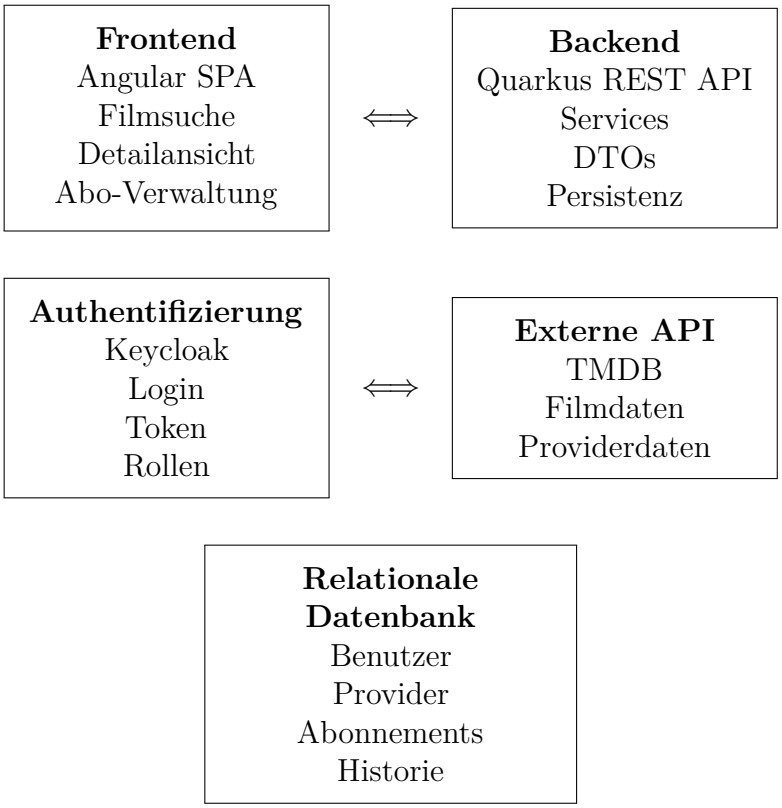

Abbildung 1: Graphische Gesamtarchitektur der Anwendung

5.2 Kommunikation zwischen Frontend und Backend

Die Kommunikation zwischen Frontend und Backend erfolgt über HTTP-basierte REST-Aufrufe. Im Frontend wurden dafür eigene Angular-Services verwendet, um Daten strukturiert von den Schnittstellen zu laden und an die Komponenten weiterzugeben [6]. Dieses Vorgehen hat den Vorteil, dass die Komponenten selbst möglichst wenig Logik für Datenzugriffe enthalten und sich stärker auf Darstellung und Interaktion konzentrieren können.

Im Projekt wurden insbesondere Services für Filmdaten, Providerdaten und die Authentifizierungsanbindung eingesetzt. Dadurch konnte die Anwendung Suchanfragen, Detailansichten und benutzerspezifische Inhalte in einer einheitlichen Struktur verarbeiten. Für das Frontend war dabei vor allem wichtig, dass externe Daten zuverlässig geladen, zwischengespeichert und in geeigneter Form in der Oberfläche dargestellt werden.

Bei benutzerspezifischen Daten reicht ein normaler HTTP-Aufruf jedoch nicht aus. Solche Daten können nur dann vom Backend geladen werden, wenn das Frontend ein gültiges Bearer-Token mitsendet. Dieses Token wird nach erfolgreicher Anmeldung über Keycloak bereitgestellt und bei geschützten Anfragen im Authorization-Header übertragen. Auf diese Weise kann das Backend prüfen, ob der Benutzer authentifiziert ist und Zugriff auf die angeforderten Daten besitzt [9, 10].

5.3 API-Integration externer Movie-Datenbanken

Ein wichtiger Bestandteil der Anwendung ist die Darstellung von Filminformationen und Streaming-Anbietern. Aus Sicht des Frontends bestand die Aufgabe darin, diese Informationen benutzerfreundlich aufzubereiten und in unterschiedlichen Ansichten darzustellen. Dazu gehören insbesondere die Filmübersicht, die Detailansicht einzelner Filme sowie die Anzeige verfügbarer Anbieter.

Die Filmdaten werden über bereitgestellte Schnittstellen geladen und in Angular-Komponenten eingebunden. Für die Benutzeroberfläche war dabei entscheidend, dass Suchergebnisse übersichtlich dargestellt, Detailinformationen verständlich aufbereitet und Navigationswechsel zwischen Listen- und Detailansicht flüssig umgesetzt werden. Da die Anwendung als Single-Page-Application realisiert wurde, konnte dieser Wechsel ohne vollständigen Seitenneuaufbau erfolgen [7]. Als externe Grundlage diente dabei

die Dokumentation der TMDB API, insbesondere für Authentifizierung, Filmdaten und Anbieterinformationen [2, 3].

Platzhalter für Frontend-Screenshot
Film-Detailansicht mit Anzeige der verfügbaren Streaming-Anbieter
Hier später Screenshot der Provider-Darstellung einfügen.

Abbildung 2: Platzhalter für einen Screenshot der Anbieteranzeige im Frontend

5.4 Sicherheits- und Datenschutzkonzept

Aus Frontend-Sicht spielt das Sicherheits- und Datenschutzkonzept vor allem bei der Zugriffskontrolle auf geschützte Bereiche eine Rolle. Öffentliche Inhalte der Anwendung können ohne Anmeldung aufgerufen werden, während persönliche Funktionen wie die Verwaltung eigener Streaming-Abonnements nur nach erfolgreicher Authentifizierung zugänglich sind. Für diesen Zweck wurde Keycloak in das Frontend integriert [9, 10].

Ein wichtiger Bestandteil dieses Konzepts ist, dass personenbezogene oder benutzerspezifische Daten nicht öffentlich abrufbar sind. Das Backend stellt diese Informationen nur dann bereit, wenn bei der Anfrage ein gültiges Bearer-Token übertragen wird. Das Frontend übernimmt dabei die Aufgabe, den Authentifizierungsstatus zu verwalten und das Token bei geschützten Requests mitzuschicken. Dadurch wird sichergestellt, dass persönliche Inhalte nur autorisierten Benutzern angezeigt werden [9].

Die Angular-Anwendung erkennt den Authentifizierungsstatus der Benutzer und steuert darauf basierend den Zugriff auf geschützte Routen. Dadurch wird sichergestellt, dass sensible benutzerspezifische Inhalte nicht ohne Anmeldung angezeigt werden. Gleichzeitig verbessert diese Struktur die Benutzerfreundlichkeit, da sich öffentliche und persönliche Bereiche klar voneinander trennen lassen. Die eigentliche Verwaltung von Zugangsdaten erfolgt dabei nicht im Frontend selbst, sondern über das externe Authentifizierungssystem.

5.5 Datenbanken

Um Dateneingaben persistent zu speichern und strukturiert zu verarbeiten, ist der Einsatz einer Datenbank unerlässlich. Eine Datenbank stellt die zentrale Instanz für die Speicherung, Organisation und Verwaltung von Daten dar. Sie ermöglicht nicht nur das einfache Ablegen von Informationen, sondern auch deren gezielte Abfrage, Aktualisierung und Löschung. Um auf die gespeicherten Daten zugreifen und diese verwalten zu können, wird ein sogenanntes Datenbankmanagementsystem (DBMS) eingesetzt, das als Vermittler zwischen der Anwendung und den physisch gespeicherten Daten fungiert.

5.5.1 Datenbank definition

Eine Datenbank ist eine strukturierte Sammlung von Daten, die so organisiert ist, dass sie effizient gespeichert, abgerufen und verwaltet werden kann. Im Gegensatz zur einfachen Dateiablage bietet eine Datenbank Mechanismen zur Vermeidung von Redundanzen, zur Sicherstellung der Datenkonsistenz sowie zur gleichzeitigen Nutzung durch mehrere Benutzer oder Anwendungen.

Grundlegende Begriffe im Datenbankumfeld sind:

- **Entität**: Ein eindeutig identifizierbares Objekt der realen Welt, das in der Datenbank abgebildet wird (z. B. ein Benutzer oder ein Streaming-Anbieter).
- **Attribut**: Eine Eigenschaft einer Entität (z. B. Name oder ID).
- **Relation**: Eine Tabelle in einer relationalen Datenbank, die Entitäten eines bestimmten Typs speichert.
- **Primärschlüssel**: Ein Attribut oder eine Kombination von Attributen, das eine Entität eindeutig identifiziert.
- **Fremdschlüssel**: Ein Attribut, das auf den Primärschlüssel einer anderen Tabelle verweist und damit Beziehungen zwischen Entitäten abbildet.

5.5.2 Datenbank Managment System - DBMS

Ein Datenbankmanagementsystem (DBMS) ist eine Software, die den Zugriff auf eine Datenbank verwaltet und kontrolliert. Es stellt die Schnittstelle zwischen der

Anwendung und den physisch gespeicherten Daten dar und übernimmt Aufgaben wie die Speicherverwaltung, die Zugriffskontrolle, die Transaktionsverwaltung sowie die Sicherstellung der Datenkonsistenz.

Ein DBMS besteht im Wesentlichen aus zwei zentralen Komponenten. Der *physische Datenbestand* umfasst die tatsächlich auf einem Datenträger gespeicherten Daten. Der *Systemkatalog* – auch Data Dictionary genannt – enthält Metainformationen über die Struktur der Datenbank, also Informationen über Tabellen, Spalten, Datentypen, Beziehungen und Zugriffsrechte. Mithilfe des Systemkatalogs kann das DBMS Anfragen korrekt interpretieren und die gespeicherten Daten strukturiert zurückgeben.

Zu den bekanntesten DBMS zählen PostgreSQL, MySQL, Oracle Database und Microsoft SQL Server im relationalen Bereich sowie MongoDB und Redis im nicht-relationalen Bereich.

5.5.3 Datenbank Komponenten

Die Physische Speicherung aka der Datenbaestand wo was gespeichert ist und der Systemkatalog mit dem ein DBMS dann auf die Datenzugreifen konnte.

5.6 Datenbankkonzept und architektonische Entscheidungen

Für das Backend der Anwendung spielt die Wahl der geeigneten Datenbanktechnologie eine zentrale Rolle. Trotz vorhandener Kenntnisse im Bereich nicht-relationaler Datenbanken (NoSQL) – wie etwa MongoDB oder Vektordatenbanken – wurde im Rahmen dieses Projekts bewusst eine relationale Datenbank eingesetzt.

Die Entscheidung für ein relationales Datenbanksystem basiert auf mehreren Faktoren. Die Anwendung weist eine klar strukturierte und logisch getrennte Datenarchitektur auf, wodurch sich die Daten und deren Beziehungen gut in einem relationalen Modell abbilden lassen. Entitäten und ihre Relationen konnten somit eindeutig definiert und konsistent strukturiert werden. Dieses Vorgehen entspricht dem etablierten Standard bei Anwendungen mit stark strukturierten Daten und klar definierten Geschäftsprozessen.

Ein weiterer wesentlicher Aspekt ist die Skalierbarkeit des Projekts. Relationale Datenbanken bieten durch Normalisierung, Transaktionssicherheit (ACID-Prinzip) und klar definierte Beziehungen eine stabile Grundlage für zukünftige Erweiterungen. Voraussetzung für eine nachhaltige Skalierbarkeit ist jedoch eine sorgfältig durchdachte Datenbankstruktur. Eine frühzeitig geplante und konsistente Modellierung – beispielsweise durch ein Entity-Relationship-Diagramm (ERD) – ist essenziell, um spätere strukturelle Anpassungen zu minimieren.

5.6.1 Herausforderungen im Entwicklungsprozess

Im Verlauf der Diplomarbeit traten mehrere strukturelle Herausforderungen auf, die im Sinne einer transparenten Dokumentation offen dargestellt werden sollen.

Zu Beginn der Implementierungsphase wurde ohne vollständig ausgearbeitetes Gesamtkonzept mit der Programmierung begonnen. Das Entity-Relationship-Diagramm wurde parallel zur Entwicklung erstellt, wodurch sich die Datenbankstruktur dynamisch entwickelte. Dieses iterative Vorgehen führte zwar zu schneller praktischer Umsetzung, hatte jedoch den Nachteil, dass strukturelle Änderungen im Nachhinein größeren Anpassungsaufwand verursachten.

Eine weitere wesentliche Herausforderung ergab sich durch die späte Integration von Keycloak als Identity- und Access-Management-Lösung. Ursprünglich war die Benutzerverwaltung eigenständig implementiert worden. Durch die nachträgliche Einführung von Keycloak mussten die bestehenden User-Strukturen grundlegend überarbeitet werden. Dies hatte zur Folge, dass die Benutzerentitäten angepasst werden mussten, die Authentifizierungs- und Autorisierungslogik neu strukturiert wurde sowie abhängige Klassen und Beziehungen refaktoriert und teilweise neu implementiert werden mussten. Diese Umstellung führte zu zusätzlichem Entwicklungsaufwand, zeigte jedoch zugleich die Bedeutung einer frühzeitigen Architekturentscheidung – insbesondere im Bereich der Sicherheits- und Authentifizierungsmechanismen.

```
+---------------------+
|        User         |
+---------------------+
| id (PK)             |
| username            |
| email               |
| password            |
| providers           |  ← String Set (e.g. "Netflix,Disney+,Prime")
| createdAt           |
+---------------------+
            |
            | 1
            |
            |------< N
                     |
             +----------------------+
             |        Movie         |
             +----------------------+
             | id (PK)              |
             | title                |
             | genre                |
             | releaseYear          |
             | providerName         | ← Stored as String (duplicate data)
             | userId (FK → User.id)|
             +----------------------+
```

Abbildung 3: Erstes Entity-Relationship-Diagramm der Anwendung

5.6.2 Modellierung der Many-to-Many-Beziehung und strukturelle Weiterentwicklung

Im weiteren Verlauf der Systemarchitektur zeigte sich deutlich der Mehrwert einer sauberen relationalen Modellierung, insbesondere bei der Abbildung von Many-to-Many-Beziehungen.

Im Kontext der vorliegenden Applikation – einem Mediatheken-Abonnement-Manager – besteht zwischen den Entitäten *User* und *Provider* keine einfache 1:n-Beziehung, sondern eine n:m-Beziehung. Ein Benutzer kann mehrere Anbieter abonnieren, während ein Anbieter von mehreren Benutzern genutzt werden kann.

Die Einführung einer eigenständigen Relationstabelle (`UserProviderSubscription`) dient hierbei nicht ausschließlich der technischen Verknüpfung zweier Fremdschlüssel. Vielmehr übernimmt diese Tabelle eine fachlich zentrale Rolle innerhalb des Domänenmodells. In der Relationstabelle werden zusätzliche Attribute gespeichert, die die konkrete Ausprägung der Beziehung beschreiben:

- Beginn des Abonnements (Startdatum)

Abbildung 4: Überarbeitetes Entity-Relationship-Diagramm nach Integration von Keycloak

- Ende des Abonnements (Enddatum)
- Zahlungszyklus (Enum: monatlich oder jährlich)

Diese Attribute gehören semantisch weder ausschließlich zum Benutzer noch ausschließlich zum Anbieter, sondern definieren die konkrete Vertragsbeziehung zwischen beiden Entitäten. Die Modellierung über eine separate Relationstabelle ermöglicht somit eine fachlich korrekte Abbildung der Geschäftslogik, die Erweiterbarkeit um weitere beziehungsbezogene Attribute – etwa Preis, Rabatt oder Kündigungsstatus – sowie eine klare Trennung der Verantwortlichkeiten innerhalb des Datenmodells. Gerade an dieser Stelle wird deutlich, dass relationale Datenbanken ihre Stärke bei strukturierten und stark beziehungsorientierten Datenmodellen ausspielen.

5.6.3 Reflexion der anfänglichen Modellierungsentscheidung

Rückblickend stellt sich die anfängliche Entscheidung, Anbieter als `Set<String>` direkt innerhalb der User-Entität zu speichern, als struktureller Fehler heraus. Diese vereinfachte Modellierung war zwar für einen ersten Prototyp funktional ausreichend, führte jedoch zu mehreren konzeptionellen Einschränkungen:

1. Es konnten keine anbieterspezifischen Metadaten gespeichert werden (z. B. Website, Kategorie oder Beschreibung).
2. Die konkrete Ausgestaltung der Beziehung – Startdatum, Zahlungszyklus etc. – war nicht modellierbar.
3. Eine konsistente Datenhaltung war nicht gewährleistet, da String-Werte fehleranfällig und nicht referenziell abgesichert sind.
4. Erweiterungen des Systems hätten tiefgreifende strukturelle Änderungen erfordert.

Insbesondere der Umstand, dass beziehungsrelevante Informationen nicht abgebildet werden konnten, verdeutlicht die Problematik dieser frühen Designentscheidung. Die Speicherung als String-Set widersprach den Grundprinzipien der Normalisierung und verhinderte eine skalierbare Weiterentwicklung des Systems. Erst durch die Einführung einer eigenständigen `Provider`-Entität sowie einer Relationstabelle konnte das Datenmodell den fachlichen Anforderungen der Applikation gerecht werden. Diese Umstrukturierung stellt einen wesentlichen architektonischen Reifeprozess im Projektverlauf dar.

5.7 Backend-Architektur

Die Backend-Architektur der vorliegenden Anwendung folgt einem mehrschichtigen Aufbau, der eine klare Trennung der Verantwortlichkeiten gewährleistet. Dieses Architekturmuster wird auch als *Layered Architecture* bezeichnet. Jede Schicht übernimmt eine klar definierte Aufgabe, wodurch die Wartbarkeit, Testbarkeit und Erweiterbarkeit der Anwendung verbessert wird. Eine eingehende HTTP-Anfrage durchläuft dabei die Resource-Schicht, wird über DTOs weitergeleitet, von der Service-Schicht verarbeitet und schließlich über das Repository auf die Datenbankentitäten abgebildet.

5.7.1 Entity

Die Entity-Schicht repräsentiert die Datenmodelle einer Anwendung. Jede Entity-Klasse bildet die Struktur einer Datenbanktabelle ab und wird mithilfe von Annotationen der Jakarta Persistence API (JPA) mit der Datenbank verknüpft. Entity-Klassen enthalten ausschließlich die Datenbankstruktur sowie einfache Zugriffsmethoden. Geschäftslogik hat in dieser Schicht keine Berechtigung, da dies die Trennung der Verantwortlichkeiten verletzen würde.

5.7.2 Repository

Die Repository-Schicht kapselt den Zugriff auf die Datenbank und stellt eine Abstraktion zwischen der Datenzugriffslogik und den darüberliegenden Schichten dar. Sie definiert die grundlegenden CRUD-Operationen (Create, Read, Update, Delete) sowie komplexere Datenbankabfragen. Durch diese Trennung bleibt die Geschäftslogik unabhängig von der konkreten Datenbankimplementierung, was einen späteren Austausch der Persistenzschicht erleichtert und die Testbarkeit der Anwendung erhöht.

5.7.3 Service

Die Service-Schicht beinhaltet die Geschäftslogik der Anwendung und fungiert als Vermittler zwischen der Resource- und der Repository-Schicht. Sie ist verantwortlich für die Verarbeitung und Transformation von Daten, die Koordination mehrerer Repository-Zugriffe sowie die Kommunikation mit externen Systemen. Die Service-Schicht stellt sicher, dass die Geschäftsregeln der Anwendung zentral und unabhängig von der Präsentations- oder Datenzugriffsschicht definiert sind.

5.7.4 Resource

Die Resource-Schicht ist für die Bereitstellung der RESTful-Endpunkte verantwortlich. Sie nimmt eingehende HTTP-Anfragen entgegen, delegiert die Verarbeitung an die zuständigen Service-Klassen und gibt die entsprechenden Antworten an den Client zurück. Die Resource-Schicht enthält dabei selbst keine Geschäftslogik, sondern fungiert ausschließlich als Schnittstelle zwischen dem Backend und den konsumierenden Clients. Durch diese strikte Trennung bleibt die Schicht schlank und leicht austauschbar.

5.7.5 DTO – Data Transfer Objects

Data Transfer Objects (DTOs) sind speziell zugeschnittene Datenobjekte, die dem gezielten Datenaustausch zwischen verschiedenen Schichten oder zwischen Backend und Client dienen. Anstatt vollständige Entity-Objekte direkt weiterzugeben, werden DTOs verwendet, die ausschließlich jene Felder enthalten, die für den jeweiligen Anwendungsfall erforderlich sind.

DTOs erfüllen dabei zwei wesentliche Zwecke. Erstens erhöhen sie die Sicherheit der Anwendung: Sensible Felder einer Entity, wie etwa Passwort-Hashes oder interne Systemfelder, werden nicht versehentlich nach außen weitergegeben. Zweitens wird eine Entkopplung zwischen der internen Datenstruktur und der nach außen sichtbaren API-Struktur erreicht. Änderungen am Datenbankmodell haben dadurch keinen direkten Einfluss auf die API-Schnittstelle, was die Stabilität der Anwendung langfristig sichert.

5.8 Implementierung des Backends in der vorliegenden Applikation

Die im vorherigen Abschnitt beschriebenen Architekturprinzipien wurden in der vorliegenden Anwendung konsequent umgesetzt. Das Backend basiert auf Quarkus und verwendet Hibernate ORM für die Datenbankanbindung. Die folgende Beschreibung erläutert, wie die einzelnen Schichten konkret implementiert wurden.

5.8.1 Entity

Die Datenbankstruktur der Anwendung wird durch fünf Entity-Klassen sowie ein Enum abgebildet, die gemeinsam das Domänenmodell der Anwendung repräsentieren.

Die zentrale Entität ist `UserMovieDB`, welche einen registrierten Benutzer der Anwendung darstellt. Als Primärschlüssel wird dabei keine automatisch generierte Datenbank-ID verwendet, sondern die `UUID` des Benutzers aus dem Identity-Provider Keycloak. Dies stellt sicher, dass die interne Benutzerrepräsentation direkt mit dem Authentifizierungssystem verknüpft ist.

Die Entität `Provider` repräsentiert einen Streaming-Anbieter wie etwa Netflix oder Amazon Prime. Neben einer internen Datenbank-ID speichert sie die `tmdbProviderId`, also die eindeutige ID des Anbieters in der TMDB API. Diese dient als Brücke zwischen den intern gespeicherten Providern und den von der externen API gelieferten Daten. Jedem Provider kann zusätzlich eine Liste von `CountryPriority`-Einträgen zugeordnet werden, die die Anzeigepriorität des Anbieters je nach Land abbilden.

Die Verknüpfung zwischen Benutzer und Streaming-Anbieter erfolgt über die Entität `UserProviderSubscription`. Diese speichert neben den Fremdschlüsseln auf `UserMovieDB` und `Provider` auch den Abrechnungszyklus des Abonnements sowie das letzte Abrechnungsdatum. Der Abrechnungszyklus wird über das Enum `BillingCycle` mit den Ausprägungen `MONTHLY` und `YEARLY` abgebildet. Eine eindeutige Kombination aus Benutzer und Anbieter wird durch eine `UniqueConstraint`-Annotation auf Datenbankebene sichergestellt, sodass ein Benutzer denselben Anbieter nicht mehrfach abonnieren kann. Die Entität stellt zudem eine berechnete Methode `getNextBillingDate()` bereit, die basierend auf dem Abrechnungszyklus das nächste Fälligkeitsdatum ermittelt.

Die Entität `History` speichert den Filmschauerverlauf eines Benutzers. Sie referenziert den jeweiligen Benutzer über einen Fremdschlüssel sowie die `tmdbMovieId` des gesehenen Films. Der Zeitpunkt des Ansehens wird automatisch beim Erstellen eines `History`-Eintrags auf den aktuellen Zeitstempel gesetzt.

5.8.2 Repository

Die Repository-Schicht der Anwendung besteht aus drei Klassen, die jeweils den Datenbankzugriff für eine bestimmte Entität kapseln.

Das `UserMovieDBRepository` implementiert das `PanacheRepository`- Interface und stellt spezialisierte Abfragemethoden bereit. Die Methode `findByUUID` sucht einen Benutzer anhand seiner Keycloak-UUID und gibt ein `Optional` zurück, um den Fall eines nicht vorhandenen Benutzers typsicher zu behandeln. Die Methode `getUserProviderTmdbIds` liefert alle TMDB-Provider-IDs der Abonnements eines bestimmten Benutzers als `Set`, das anschließend im Service-Layer für den Abgleich mit der TMDB API verwendet wird. Besonders hervorzuheben ist die Methode `getProvidersWithOwnership`, die mithilfe einer JPQL-Abfrage alle Provider aus der Datenbank lädt und dabei über einen `LEFT JOIN` mit den Abonnements des Benutzers verknüpft. Das Ergebnis enthält für jeden

Provider ein Flag, das angibt, ob der Benutzer diesen abonniert hat. Dadurch kann das Frontend mit einem einzigen Request alle relevanten Informationen abrufen. Darüber hinaus enthält das Repository die Methode `updateSubscriptions`, die die Abonnements eines Benutzers anhand einer übergebenen Liste von `SubscriptionUpdateDTO`-Objekten aktualisiert. Dabei werden neue Abonnements angelegt, bestehende aktualisiert und nicht mehr enthaltene entfernt.

Das `ProviderRepository` verwaltet die Streaming-Anbieter in der Datenbank. Die zentrale Methode ist `upsertFromTmdb`, die beim Anwendungsstart aufgerufen wird: Sie prüft, ob ein Provider mit der gegebenen `tmdbProviderId` bereits existiert, und aktualisiert dessen Daten oder legt einen neuen Eintrag an. Dadurch wird eine konsistente Synchronisierung der Provider-Daten mit der TMDB API gewährleistet.

Das `HistoryRepository` implementiert ebenfalls das `PanacheRepository`-Interface und stellt Methoden für den Zugriff auf den Filmschauerverlauf bereit. Die Methode `findByUserId` gibt alle History-Einträge eines Benutzers sortiert nach dem Zeitpunkt des Ansehens zurück. Über `findByUserIdLimit` kann die Anzahl der zurückgegebenen Einträge begrenzt werden, was eine einfache Paginierung ermöglicht. Die Methode `hasWatchedMovie` prüft, ob ein Benutzer einen bestimmten Film bereits gesehen hat.

5.8.3 Service

Die Service-Schicht der Anwendung besteht aus drei Klassen, die jeweils einen klar abgegrenzten Verantwortungsbereich abdecken.

Der `UserService` ist für die Verwaltung des aktuell authentifizierten Benutzers zuständig. Er liest die Keycloak-UUID aus dem JWT-Token aus und prüft über das `UserMovieDBRepository`, ob der Benutzer bereits in der Datenbank existiert. Ist dies nicht der Fall, wird automatisch ein neuer `UserMovieDB`-Eintrag angelegt. Dieses Muster wird als *Lazy User Creation* bezeichnet und stellt sicher, dass beim ersten authentifizierten Zugriff eines neuen Benutzers automatisch ein Datenbankprofil erstellt wird, ohne dass eine explizite Registrierung erforderlich ist.

Der `TmdbService` kapselt die gesamte Kommunikation mit der externen TMDB API. Die HTTP-Anfragen werden über die Bibliothek OkHttp durchgeführt, die empfangenen JSON-Antworten mit Jackson geparst und auf DTOs abgebildet. Der API-Schlüssel wird nicht im Quellcode hinterlegt, sondern über den Quarkus-Konfigurationsmechanismus

mittels `@ConfigProperty` injiziert, was die Sicherheit der Anwendung erhöht. Der Service stellt Methoden zum Abrufen von Filminformationen per ID oder Suchbegriff, zum Laden verfügbarer Streaming-Anbieter sowie zum Abrufen trendender Filme bereit. Die Methode `getFilteredProvidersForUser` kombiniert dabei externe API-Daten mit internen Datenbankdaten: Sie ruft die verfügbaren Provider eines Films von der TMDB API ab, gleicht diese mit den vom Benutzer abonnierten Anbietern aus der Datenbank ab und setzt das Flag `ownedByUser` im zurückgegebenen `ProviderInfoDTO`. Die Ergebnisliste wird abschließend so sortiert, dass die vom Benutzer abonnierten Anbieter an erster Stelle erscheinen.

Der `ProviderSyncService` wird beim Start der Anwendung automatisch ausgeführt, indem er auf das Quarkus-Lifecycle-Event `StartupEvent` reagiert. Er ruft über den `TmdbService` alle verfügbaren Streaming-Anbieter von der TMDB API ab und synchronisiert diese über die `upsertFromTmdb`-Methode des `ProviderRepository` in die lokale Datenbank. Dadurch ist sichergestellt, dass die Anwendung stets eine aktuelle Liste aller Provider vorhält, ohne dass ein manueller Synchronisierungsschritt erforderlich ist. Der gesamte Vorgang wird in einer Transaktion ausgeführt, was durch die `@Transactional`-Annotation sichergestellt wird.

5.8.4 Resource

Die Resource-Schicht der Anwendung besteht aus drei produktiven Resource-Klassen, die die REST-Endpunkte des Backends definieren.

Die `MovieResource` stellt Endpunkte unter dem Basispfad `/api/movies` bereit. Sie ermöglicht das Abrufen von Filmdaten anhand einer ID oder eines Suchbegriffs, das Laden verfügbarer Streaming-Anbieter für einen Film sowie das Abrufen trendender Filme. Einige Endpunkte sind öffentlich zugänglich und mit `@PermitAll` annotiert, während der Endpunkt `/api/movies/{id}/providers/for-user` eine gültige Authentifizierung erfordert und über `@RolesAllowed` auf die Rollen `user` und `medien-abo-users-role` beschränkt ist. Bei diesem Endpunkt wird die Benutzer-ID direkt aus dem JWT-Token extrahiert, sodass keine User-ID in der URL übermittelt werden muss.

Die `UserResource` stellt Endpunkte unter dem Basispfad `/api/users` bereit und ist vollständig authentifizierungsgeschützt. Sie bietet Funktionalitäten zur Verwaltung des eigenen Benutzerprofils, der Streaming-Abonnements sowie der Filmschauhistorie. Über `GET /api/users/me` kann der aktuelle Benutzer seine Profildaten abrufen, wobei bei

einem erstmaligen Zugriff automatisch ein Datenbankprofil angelegt wird. Der Endpunkt `PUT /api/users/me/subscriptions` ermöglicht das vollständige Aktualisieren der Abonnements in einem einzigen Request. Die History-Endpunkte erlauben das Hinzufügen, Abrufen und Löschen von Einträgen im Filmschauerverlauf. Beim Löschen wird explizit geprüft, ob der Eintrag dem authentifizierten Benutzer gehört, bevor die Operation ausgeführt wird.

Die `ProviderRessource` stellt Endpunkte unter `/api/providers` bereit. Sie ermöglicht das Abrufen aller in der Datenbank gespeicherten Streaming-Anbieter sowie das manuelle Auslösen einer Synchronisierung mit der TMDB API über den Endpunkt `GET /api/providers/sync`.

5.8.5 DTO – Data Transfer Objects

In der vorliegenden Anwendung werden DTOs konsequent eingesetzt, um die Kommunikation zwischen den Schichten zu strukturieren und interne Datenbankstrukturen zu kapseln.

Der `ProviderInfoDTO` wird bei der Abfrage von Streaming-Anbietern eines Films verwendet. Er enthält die TMDB-Provider-ID, den Anbieternamen, die URL des Provider-Logos sowie das Flag `ownedByUser`, das angibt, ob der jeweilige Benutzer diesen Anbieter abonniert hat. Die vollständige `Provider`-Entity sowie interne Datenbankfelder werden dabei nicht weitergegeben.

Der `ProviderWithOwnership`-DTO wird vom `UserMovieDBRepository` direkt über eine JPQL-Konstruktorabfrage befüllt und an den Client übermittelt. Er enthält neben den Provider-Stammdaten auch den Abrechnungszyklus und das letzte Abrechnungsdatum des Abonnements, sofern der Benutzer den jeweiligen Anbieter abonniert hat.

Der `SubscriptionUpdateDTO` dient als Eingabe-DTO für den `PUT /api/users/me/subscriptions` Endpunkt. Er enthält die TMDB-Provider-ID, den gewünschten Abrechnungszyklus sowie das Abrechnungsdatum und ermöglicht so die vollständige Aktualisierung eines Abonnements in einem einzigen Objekt.

Der `TrendingMovieDTO` bündelt alle relevanten Informationen eines trendenden Films – Titel, Poster-URL, Kurzbeschreibung, Erscheinungsdatum, Popularitätswert sowie Bewertungsdurchschnitt und Bewertungsanzahl – in einem kompakten Objekt, das direkt an den Client übermittelt wird.

Der `HistoryDTO` dient der strukturierten Rückgabe von History-Einträgen an den Client. Er wird über eine statische `from(History)`-Methode aus der gleichnamigen Entity erstellt und stellt sicher, dass keine internen JPA-Proxies oder lazy-geladenen Assoziationen versehentlich serialisiert werden.

Der `ProviderApi`-Record dient als internes Transferobjekt zwischen dem `TmdbService` und dem `ProviderSyncService`. Er bildet die Struktur der TMDB-API-Antwort ab und ermöglicht eine typsichere Weiterverarbeitung der Rohdaten, bevor diese in die Datenbank geschrieben werden.

6 Umsetzung der Anwendung

6.1 Implementierung der Abo-Verwaltung

Ein wesentlicher Bestandteil meines Projektteils war die Umsetzung der Benutzeroberfläche zur Verwaltung persönlicher Streaming-Abonnements. Ziel war es, eine übersichtliche und einfach bedienbare Ansicht zu entwickeln, in der Benutzer ihre aktiven Anbieter verwalten und Änderungen direkt speichern können. Die technische Umsetzung erfolgte im Angular-Frontend mit einer eigenständigen Komponente für die Providerverwaltung sowie zugehörigen Services für die Kommunikation mit dem Backend [4, 6].

6.1.1 Aufbau der Provider-Verwaltung

Die Provider-Verwaltung wurde als eigener geschützter Bereich innerhalb der Angular-Anwendung umgesetzt. Benutzer können dort Streaming-Anbieter laden, durchsuchen und auswählen. Zusätzlich ist erkennbar, welche Anbieter bereits im Profil gespeichert sind. Diese Struktur verbessert die Übersichtlichkeit und ermöglicht eine klare Trennung zwischen allgemeinen Filmfunktionen und persönlichen Abo-Daten.

6.1.2 Bearbeitung und Speicherung von Abonnements

In der Oberfläche können Benutzer Anbieter als aktiv markieren und zusätzliche Angaben wie Abrechnungszyklus und Datum der letzten Abbuchung festlegen. Diese Eingaben werden im Frontend verarbeitet und anschließend gesammelt an das Backend übermittelt. Die Verwendung von Formularlogik und Datenbindung in Angular erleichtert dabei die direkte Verknüpfung zwischen Eingabefeldern und angezeigten Daten [6]. Dadurch konnten Änderungen in der Oberfläche unmittelbar sichtbar gemacht werden.

6.1.3 Filter- und Suchfunktionen in der Oberfläche

Damit die Verwaltung vieler Anbieter benutzerfreundlich bleibt, wurde die Oberfläche um Such- und Filterfunktionen ergänzt. Benutzer können nach Anbietern suchen oder sich nur bereits aktivierte Abonnements anzeigen lassen. Diese Interaktionsmöglichkeiten verbessern die Nutzbarkeit der Anwendung deutlich, da relevante Einträge schneller gefunden und bearbeitet werden können.

Platzhalter für Frontend-Screenshot
Abo-Verwaltung mit Filter- und Bearbeitungsfunktionen
Hier später Screenshot der Providerverwaltung einfügen.

Abbildung 5: Platzhalter für einen Screenshot der Abo-Verwaltung

6.2 Darstellung von Laufzeiten und Kündigungszeitpunkten

Neben der Auswahl von Streaming-Anbietern war auch die Darstellung zeitlicher Informationen ein wichtiger Teil der Abo-Verwaltung. Aus Frontend-Sicht ging es dabei vor allem darum, Eingabemöglichkeiten für abrechnungsrelevante Daten bereitzustellen und diese verständlich in die Oberfläche einzubinden. Die Benutzer sollen dadurch ihre gespeicherten Abonnements besser nachvollziehen können.

6.2.1 Erfassung abrechnungsrelevanter Daten

Für jedes aktive Abonnement können im Frontend Informationen wie Abrechnungszyklus und Datum der letzten Abbuchung erfasst werden. Diese Werte werden in Formularfeldern dargestellt und vom Benutzer bearbeitet. Die Oberfläche unterstützt dadurch nicht nur die Auswahl eines Anbieters, sondern auch die strukturierte Verwaltung der wichtigsten Abo-Metadaten.

6.2.2 Grenzen des aktuellen Umsetzungsstandes

Im aktuellen Projektstand liegt der Schwerpunkt auf der Erfassung und Speicherung dieser Daten. Eine vollständige Logik zur Berechnung von Kündigungsfristen, Erinnerungen oder automatischen Benachrichtigungen wurde im Frontend noch nicht vollständig ausgebaut. Für die schriftliche Arbeit ist es daher sinnvoll, zwischen der bereits umgesetzten Datenerfassung und möglichen zukünftigen Erweiterungen zu unterscheiden.

6.3 Verknüpfung von Filmen mit Streaming-Anbieter

Ein zentraler Mehrwert der Anwendung besteht darin, dass Filmrecherche und Abo-Verwaltung in einer Oberfläche zusammengeführt werden. Im Frontend wurde dafür eine Benutzeroberfläche entwickelt, in der Benutzer Filme suchen, Trending-Inhalte ansehen und Detailinformationen zu einzelnen Filmen aufrufen können. Die Verknüpfung mit Streaming-Anbietern ermöglicht es, Filminformationen nicht isoliert, sondern im Zusammenhang mit den eigenen Abonnements zu betrachten.

6.3.1 Umsetzung der Filmsuche

Die Filmsuche wurde als eigener Bereich im Frontend umgesetzt. Benutzer können nach Filmtiteln suchen und erhalten eine Liste passender Ergebnisse, die dynamisch geladen und angezeigt werden. Da das Frontend als Single-Page-Application realisiert wurde, bleiben Suchanfragen und Seitenwechsel innerhalb derselben Anwendung und wirken dadurch deutlich flüssiger [7, 5].

6.3.2 Darstellung von Trending-Filmen

Zusätzlich zur klassischen Suche wurde eine Ansicht für Trending-Filme integriert. Dadurch können Benutzer aktuelle oder populäre Inhalte entdecken, ohne gezielt nach einem Titel suchen zu müssen. Diese Funktion erweitert die Anwendung um einen explorativen Aspekt und erhöht den praktischen Nutzen des Frontends.

6.3.3 Detailansicht einzelner Filme

Für einzelne Filme wurde eine eigene Detailansicht umgesetzt. Dort werden weiterführende Informationen wie Titel, Beschreibung, Erscheinungsjahr, Laufzeit, Genres und weitere Metadaten dargestellt. Die Aufbereitung dieser Daten im Frontend war wichtig, damit die Benutzer nicht nur Suchergebnisse sehen, sondern Inhalte auch genauer beurteilen können.

6.3.4 Verbindung zur Anbieteranzeige

Ein besonderer Fokus lag auf der Darstellung von Streaming-Anbietern im Zusammenhang mit Filminformationen. Dadurch kann die Anwendung anzeigen, auf welchen Plattformen ein Film verfügbar ist. Aus Sicht des Frontends bestand die Aufgabe vor allem darin, diese Informationen übersichtlich darzustellen und sinnvoll in die Filmansichten zu integrieren. Diese Verbindung zwischen Inhalt und Verfügbarkeit stellt den eigentlichen Kernnutzen des Projekts dar.

Platzhalter für Frontend-Screenshot
Filmsuche oder Trending-Ansicht
Hier später Screenshot der Suche oder Startseite einfügen.

Abbildung 6: Platzhalter für einen Screenshot der Filmsuche oder Trending-Ansicht

Platzhalter für Frontend-Screenshot
Detailansicht eines Films mit Metadaten
Hier später Screenshot der Film-Detailseite einfügen.

Abbildung 7: Platzhalter für einen Screenshot der Film-Detailansicht

6.4 Account- und Passwortverwaltung

Die Benutzerverwaltung wurde im Frontend nicht als Eigenlösung umgesetzt, sondern über die Integration von Keycloak realisiert. Dadurch konnte auf eine eigene Passwortspeicherung verzichtet werden, während gleichzeitig ein professioneller und sicherer Anmeldeprozess in die Anwendung eingebunden wurde [9, 10]. Für meinen Projektteil war vor allem relevant, wie sich Login, Registrierung und geschützte Routen in die Angular-Anwendung integrieren lassen.

6.4.1 Einbindung von Login und Registrierung

Im Frontend wurden Login- und Registrierungsfunktionen so eingebunden, dass Benutzer bei Bedarf an das Keycloak-System weitergeleitet werden. Nach erfolgreicher Anmeldung kehren sie in die Anwendung zurück und können geschützte Bereiche verwenden. Dadurch bleibt der Authentifizierungsprozess klar vom restlichen Frontend getrennt und dennoch benutzerfreundlich in die Anwendung eingebettet.

6.4.2 Geschützte Bereiche im Frontend

Bestimmte Bereiche, insbesondere die Verwaltung persönlicher Abonnements, sind nur für angemeldete Benutzer zugänglich. Das Frontend prüft dafür den Authentifizierungsstatus und steuert den Zugriff auf geschützte Routen. Dadurch wird verhindert, dass persönliche Funktionen ohne gültige Anmeldung aufgerufen werden können.

6.4.3 Verwendung von Bearer-Token bei geschützten Requests

Ein wichtiger technischer Punkt der Umsetzung war, dass benutzerspezifische Daten nicht ohne Authentifizierung geladen werden können. Wenn das Frontend persönliche Informationen vom Backend anfordert, muss ein gültiges Bearer-Token mitgesendet werden. Dieses Token wird nach erfolgreicher Anmeldung über Keycloak bereitgestellt und bei geschützten HTTP-Anfragen übertragen [9]. Erst dadurch kann das Backend prüfen, ob der Benutzer berechtigt ist, auf die jeweiligen Daten zuzugreifen.

6.4.4 Herausforderungen bei der Integration von Keycloak

Die Integration von Keycloak in Angular war mit mehreren Herausforderungen verbunden. Dazu gehörten insbesondere die korrekte Weiterleitung nach dem Login, die Behandlung geschützter Routen sowie die sichere Übergabe des Bearer-Tokens bei Anfragen an das Backend. Diese Punkte waren im Frontend besonders relevant, da die Anwendung nur dann benutzerfreundlich funktioniert, wenn Anmeldung, Rückleitung und Zugriffsschutz zuverlässig zusammenspielen. Die Auseinandersetzung mit diesen Problemen war daher ein wichtiger Teil meines praktischen Projektbeitrags.

6.4.5 Analyse bestehender Lösungen

Im Zusammenhang mit Benutzerkonten stellte sich die Frage, ob zusätzlich eine eigene Passwortverwaltung innerhalb der Anwendung umgesetzt werden sollte. Bei der Betrachtung bestehender Lösungen wie Bitwarden oder 1Password wird jedoch deutlich, dass sich professionelle Passwortverwaltung auf Sicherheit, Verschlüsselung und geräteübergreifende Synchronisation spezialisiert [10]. Diese Zielsetzung unterscheidet sich klar vom Schwerpunkt des MedienAbonnementManagers.

6.4.6 Sicherheits- und Datenschutzbetrachtung

Eine selbst entwickelte Verwaltung von Passwörtern oder besonders sensiblen Zugangsdaten hätte hohe Anforderungen an Datenschutz und Sicherheit gestellt. Da der Fokus meines Projektteils auf der Frontend-Entwicklung lag, wurde bewusst auf eine eigene Passwortlösung verzichtet. Stattdessen war es sinnvoller, ein bestehendes Authentifizierungssystem einzubinden und die Anwendung auf ihre eigentlichen Kernfunktionen zu konzentrieren.

6.4.7 Entscheidung: Eigenlösung vs. Integration

Die Entscheidung fiel daher zugunsten einer Integration von Keycloak und gegen eine selbst entwickelte Passwortverwaltung. Diese Lösung reduziert die Komplexität im Frontend, verbessert die Sicherheit und ermöglicht eine klarere Trennung zwischen Benutzeroberfläche und Authentifizierungslogik. Für das Projekt war diese Entscheidung sowohl technisch als auch organisatorisch sinnvoll.

Platzhalter für Frontend-Screenshot
Login oder geschützter Bereich nach erfolgreicher Anmeldung
Hier später Screenshot zur Keycloak-Integration einfügen.

Abbildung 8: Platzhalter für einen Screenshot des Login-Flows oder eines geschützten Bereichs

6.5 Herausforderungen und Reflexion der Frontend-Entwicklung

Im Verlauf der Frontend-Umsetzung zeigte sich, dass nicht nur die reine Darstellung von Oberflächen relevant ist, sondern vor allem die saubere Verbindung zwischen Benutzerführung, Authentifizierung und Datenlogik. Gerade dort entstanden mehrere Erkenntnisse, die für den Projektverlauf und für die eigene fachliche Weiterentwicklung wesentlich waren.

6.5.1 Herausforderungen bei der Integration von Keycloak in Angular

Eine der größten Herausforderungen meines Projektteils war die Integration von Keycloak in das Angular-Frontend. Ein praktisches Problem dabei war, dass für aktuelle Angular-Versionen vergleichsweise wenig gut strukturierte und direkt übertragbare Beispiele verfügbar waren. Viele gefundene Lösungen bezogen sich auf ältere Projektstände, unvollständige Konfigurationen oder andere Framework-Kombinationen. Dadurch war es notwendig, die Einbindung von Login, Redirect, Token-Verwaltung und geschützten Routen schrittweise selbst nachzuvollziehen und an die eigene Anwendung anzupassen.

6.5.2 Herausforderungen bei der Umsetzung von Backend-Daten in der Benutzeroberfläche

Ein weiterer anspruchsvoller Punkt war die Übersetzung der vom Backend gelieferten Informationen in eine für Benutzer logische und leicht verständliche Oberfläche. Die gelieferten Daten mussten nicht nur technisch korrekt angezeigt, sondern auch sinnvoll gruppiert, gefiltert und mit Interaktionen verknüpft werden. Besonders bei Providerdaten, Benutzerabonnements und filmbasierten Anbieterinformationen war es wichtig, die Datenstruktur aus Services und DTOs so in die UI zu übertragen, dass die Oberfläche konsistent bleibt und Benutzer die Zusammenhänge schnell erfassen können.

6.5.3 Reflexion der anfänglichen Frontend-Entscheidungen

Rückblickend wurde deutlich, dass eine frühe Oberflächenidee nicht automatisch auch die beste Struktur für die spätere Weiterentwicklung darstellt. Zu Beginn lag der Fokus stark darauf, einzelne Funktionen möglichst schnell sichtbar zu machen. Erst im Verlauf der Umsetzung zeigte sich, wie wichtig eine klare Trennung zwischen Komponenten, Services, Zuständen und geschützten Bereichen ist. Diese Erkenntnis war für die Frontend-Entwicklung ähnlich wertvoll wie die Datenbank-Reflexion im Backend, weil sie gezeigt hat, dass gute Wartbarkeit nicht nur durch funktionierenden Code entsteht, sondern durch eine durchdachte Oberflächen- und Zustandsstruktur.

6.5.4 Erkenntnisgewinn für den weiteren Projektverlauf

Die Auseinandersetzung mit diesen Frontend-Problemen hat wesentlich dazu beigetragen, die Anwendung systematischer weiterzuentwickeln. Insbesondere die Keycloak-Integration und die saubere Abbildung von Backend-Daten auf UI-Komponenten haben gezeigt, dass moderne Webentwicklung nicht nur aus Design und API-Aufrufen besteht, sondern ein durchgängiges Verständnis von Authentifizierung, Datenfluss und Benutzerführung erfordert. Gerade dieser Lernprozess stellt einen wichtigen Teil meines persönlichen Beitrags zur Diplomarbeit dar.

7 Projektverlauf und Entscheidungen

7.1 Projektplanung und Arbeitsweise

7.2 Technische und fachliche Herausforderungen

7.3 Entscheidungsfindung im Projektteam

7.4 Einbindung von Betreuern und Feedback

7.5 Reaktion auf Fragen und Kritik aus Präsentationen

8 Ergebnisse und Evaluation

8.1 Erfüllung der gesetzten Ziele

8.2 Bewertung des Nutzens für Endanwender

8.3 Reflexion der eigenen Leistung

9 Ausblick

9.1 Mögliche Erweiterungen der Anwendung

9.2 Zukunftspotenzial der Plattform

10 Zusammenfassung

11 Quellen-, Literatur- und Abbildungsverzeichnis

11.1 Quellen

11.2 Literatur

11.3 Abbildungsverzeichnis

Literaturverzeichnis

[1] Deloitte, „2025 Digital Media Trends," https://www2.deloitte.com/us/en/insights/industry/technology/digital-media-trends-consumption-habits-survey/2025.html, 2025, Zugriff am 03.04.2026.

[2] The Movie Database, „Getting Started," https://developer.themoviedb.org/docs/getting-started, 2026, Zugriff am 03.04.2026.

[3] ——, „Watch Providers," https://developer.themoviedb.org/reference/tv-season-watch-providers, 2026, Zugriff am 03.04.2026.

[4] Angular Team, „What is Angular?" https://angular.dev/overview, 2026, Zugriff am 19.03.2026.

[5] ——, „Angular Routing," https://angular.dev/guide/routing, 2026, Zugriff am 19.03.2026.

[6] ——, „Forms," https://angular.dev/guide/forms, 2026, Zugriff am 19.03.2026.

[7] Mozilla Developer Network, „Single-page application," https://developer.mozilla.org/en-US/docs/Glossary/SPA, 2026, Zugriff am 19.03.2026.

[8] A. Freeman, *Pro Angular*. Apress, 2019.

[9] Keycloak Team, „Keycloak Documentation," https://www.keycloak.org/documentation, 2026, Zugriff am 19.03.2026.

[10] ——, „Keycloak," https://www.keycloak.org/, 2026, Zugriff am 19.03.2026.

[11] Tailwind Labs, „Tailwind CSS Documentation," https://tailwindcss.com/docs, 2026, Zugriff am 19.03.2026.

[12] ——, „Tailwind CSS," https://tailwindcss.com/, 2026, Zugriff am 19.03.2026.

[13] GitHub, Inc., „Git workflows," https://docs.github.com/en/get-started/getting-started-with-git/git-workflows, 2026, Zugriff am 19.03.2026.

Abbildungsverzeichnis

1 Graphische Gesamtarchitektur der Anwendung 14
2 Platzhalter für einen Screenshot der Anbieteranzeige im Frontend . . . 16
3 Erstes Entity-Relationship-Diagramm der Anwendung 20
4 Überarbeitetes Entity-Relationship-Diagramm nach Integration von Keycloak 21
5 Platzhalter für einen Screenshot der Abo-Verwaltung 31
6 Platzhalter für einen Screenshot der Filmsuche oder Trending-Ansicht . 33
7 Platzhalter für einen Screenshot der Film-Detailansicht 33
8 Platzhalter für einen Screenshot des Login-Flows oder eines geschützten Bereichs 36

Tabellenverzeichnis

Quellcodeverzeichnis